
Bölüm-01

JAVA 101

Java API'leri

JDK çeşitli programlama amaçlarına yönelik yüzlerce sınıf sunmaktadır

- Temel veri tiplerini ifade etmek için `String`, `LocalDateTime`, `BigDecimal` vs.
- Koleksiyonları işlemek için `Enumeration`, `ArrayList`, `HashMap` vs.
- Genel davranışların ele alınması ve sistem eylemleri için, `System`, `Object`, `Class` vs.
- I/O operasyonları, `FileInputStream`, `FileOutputStream` vs.
- Veritabanlarına erişim, eşzamanlılığı yönetmek, ağ iletişimlerini etkinleştirmek, scriptleri yürütmek, transaction yönetimi, güvenlik, log kaydı, grafik kullanıcı arayüzleri oluşturmak vb. için diğer birçok API sınıfı kullanılır.



API (Application Programming Interface) genel bir amaca hizmet etmek için tasarlanmış sınıflar koleksiyonunu tanımlayan bir terimdir.



Java Anahtar Kelimeler, Rezerve Kelimeler ve Bir Özel Tanımlayıcı

1.0' dan beri kullanımda

Kullanımı yok

1.2' de eklendi

1.4' te eklendi

5.0' da eklendi

9.0' da eklendi

Literal değerler için rezerve

10' da eklendi

if	implements	boolean	assert
else	extends	try	enum
continue	interface	catch	module
break	class	finally	requires
for	static	throw	transitive
do	final	throws	exports to
while	return	new	uses
switch	transient	this	provides
case	void	super	with
default	byte	instanceof	opens to
private	short	native	
protected	int	synchronized	true
public	long	volatile	false
import	char	goto	null
package	float	const	
abstract	double	strictfp	var

✓ Anahtar kelimeler ve Literaller tanımlayıcı olarak kullanılamaz (Sınıf isimleri, değişkenler, metotlar vs.)

Java dilinin alt kümeleri

<i>Temel Tipler</i>
int
long
double
char
String
boolean

<i>noktalama</i>
{
}
(
)
,
;

<i>Sayısal ifade operasyonları</i>
+
-
*
/
%
++
--

<i>karşılaştırma</i>
<
<=
>
>=
==
!=

<i>String operasyonları</i>
+
""
length()
charAt()
compareTo()
matches()

<i>boolean operasyonlar</i>
true
false
!
&&

<i>atama</i>
=
<i>flow control</i>
if
else
for
while

<i>diziler</i>
a[]
length
new

<i>Nesne Tabanlı</i>
static
class
public
private
new
final
toString()
main()

<i>type dönüşüm metodları</i>
Integer.parseInt()
Double.parseDouble()

<i>Math methodları</i>
Math.sin()
Math.cos()
Math.log()
Math.exp()
Math.pow()
Math.sqrt()
Math.min()
Math.max()
Math.abs()
Math.PI



<i>System methodları</i>
System.print()
System.println()
System.printf()

<i>our Std methods</i>
StdIn.read*()
StdOut.print*()
StdDraw.*()
StdAudio.*()
StdRandom.*()

Java'da İsimlendirme Kuralları

- Java case-sensitive bir dildir, Dog değişkeni ile dog değişkeni aynı şey değildir.
- Package isimlendirmesinde şirket veya organizasyon yapısına göre genelden özele doğru isimlendirme yapılır ve küçük harfler kullanılır. Ör: *college.staff.ce*
- Sınıf adlar için «isim» kullanılmalıdır. İlk harf ve birleşik kelime ise her kelimenin ilk harfi büyük yazılmalıdır.
- Değişken isimleri küçük harfle başlamalı, birleşik kelime ise sonraki kelimelerin ilk harfleri büyük olacak şekilde oluşturulmalıdır.
- İsimler rakamla, _ veya \$ ile başlamamalıdır.
- Constant isimleri tamamı büyük ve kelimeler arasında _ olacak şekilde isimlendirilmelidir.
- Metot isimlerine fiiller kullanılmalı, küçük harfle başlayıp diğer kelimelerin ilk harfleri büyük yazılmalıdır.

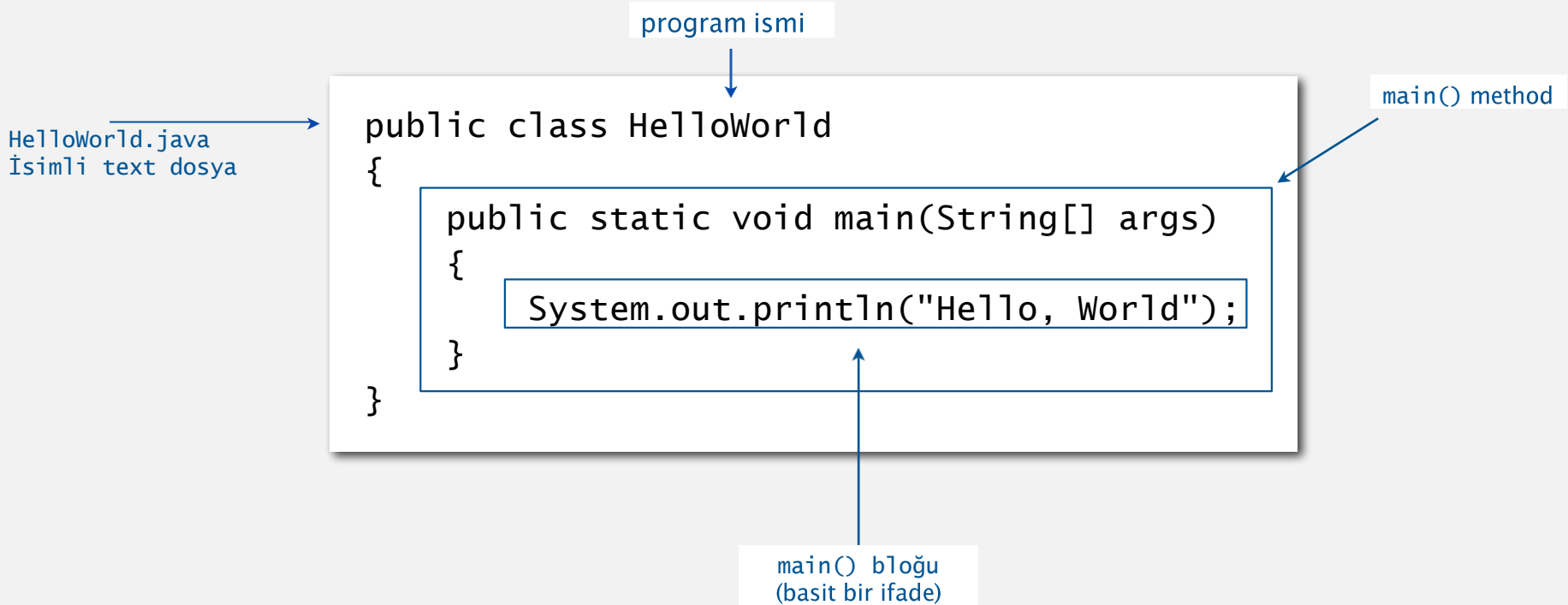
```
package : com.oracle.demos.animals  
class   : ShepherdDog  
variable: shepherdDog  
constant: MIN_SIZE  
method  : havla
```



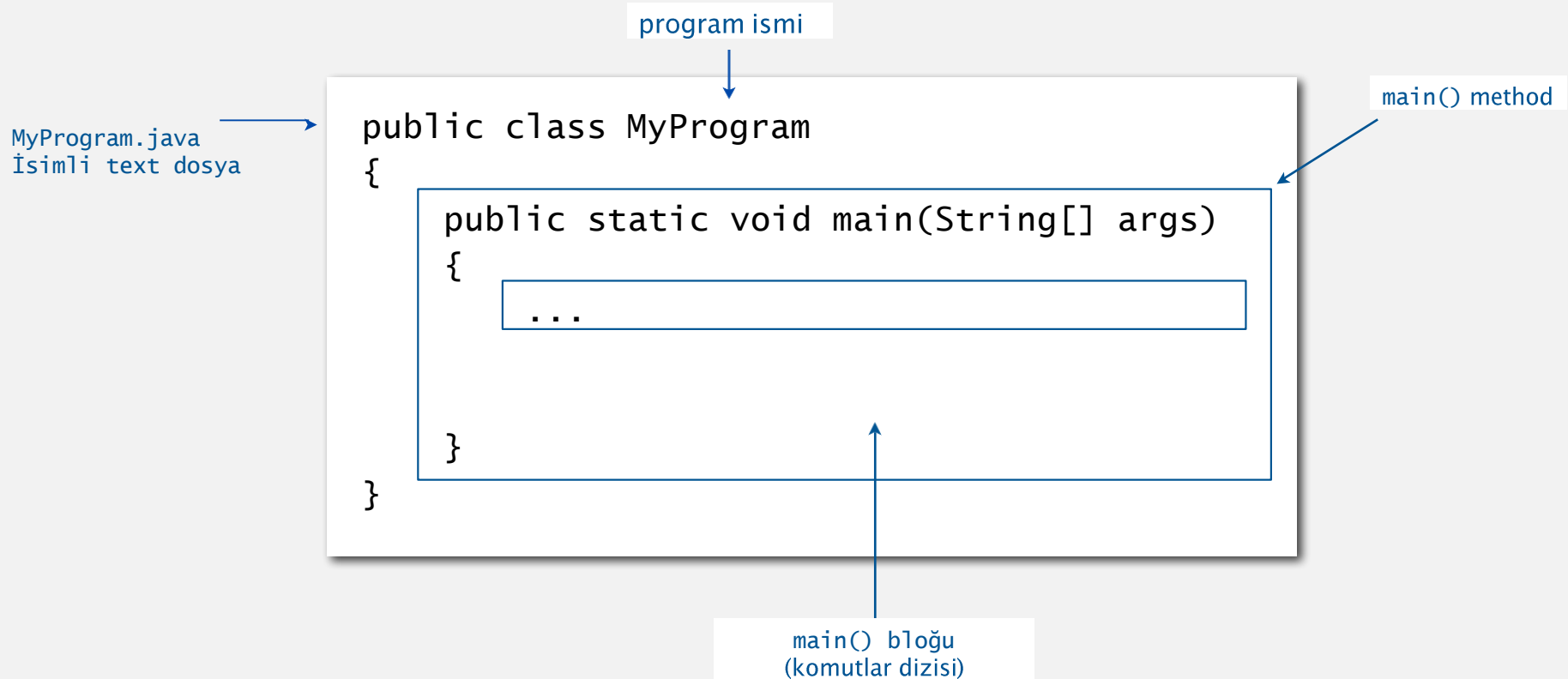
```
package : animals  
class   : Shepherd Dog  
variable: _price  
constant: minSize  
method  : xyz
```



İlk programınızın anatomisi



Diğer programlarınızın anatomisi



Aynı programın üç versiyonu

```
public class HelloWorld
{
    public static void main(String[] args)
    {
        System.out.println("Hello, World");
    }
}
```

```

/*****
 *  Compilation:  javac HelloWorld.java
 *  Execution:   java HelloWorld
 *
 *  Prints "Hello, World". By tradition, this is everyone's first program.
 *
 *  % java HelloWorld
 *  Hello, World
 *
 *****/

public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello, World");
    }
}
```



```
public class HelloWorld { public static void main(String[] args) { System.out.println("Hello, World"); } }
```

Sonuç: Fontlar, renk, yorumlar, ve extra boşluklar Java diliyle alakalı değildir..

Java da program geliştirme

Üç adımlı bir işlemdir, *feedback içerir.*

1. Programını **YAZ**

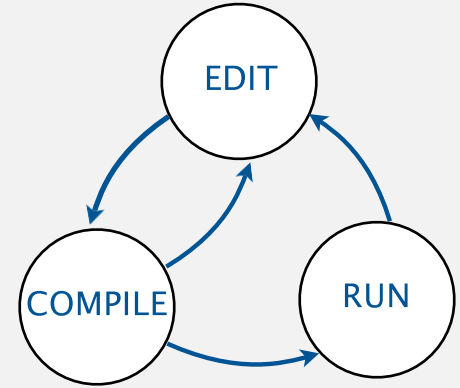
- Bilgisayar klavyesi ile kodları gir.
- Sonuç: HelloWorld.java benzeri bir dosya oluşur.

2. Çalıştırılabilir bir dosya elde etmek için programını **DERLE**

- Java Compiler kullan
- Sonuç: Bir Java bytecode dosyası HelloWorld.class
- Hata? 1. Adıma dön düzelt ve tekrar derle.

3. Programını **ÇALIŞTIR**

- Java runtime kullan.
- Sonuç: Programının çıktısı.
- Hata? 1.Adıma git düzelt, yeniden derle ve çalıştır



Temel veri tipleri

Koşul ifadeleri

Diziler ve Döngüler

Metotlar

Temel Veri Tipleri

Bir **veri tipi** birtakım değerler ve bu değerler üzerindeki operasyonlardır.

Whole numbers			
byte	short	int	long
8 bits	16 bits	32 bits	64 bits
-128	-32,768	-2,147,483,648	-9,223,372,036,854,780,000
127	32,767	2,147,483,647	9,223,372,036,854,780,000
default value 0 or 0L			
Binary 0b1001 Octal 072 Decimal 1234 Hex 0x4F Upper or lowercase L at the end indicates long value			

Floating point numbers	
float	double
32 bits	64 bits
1.4E-45	4.9E-324
3.4028235E+38	1.7976931348623157E+308
default value 0.0F or 0.0	
Normal 123.4 or exponential notations 1.234E2; Upper or lowercase F at the end indicates float value	

Boolean
boolean
default value false
true or false

Character (represents single character value)
char
16 bits
0
65,535
default value '\u0000'
Character 'A' ASCII code '\101' Unicode '\u0041' Escape Sequences: tab '\t' backspace '\b' new line '\n' carriage return '\r' form feed '\f' single quote \' double quote \" backslash \\

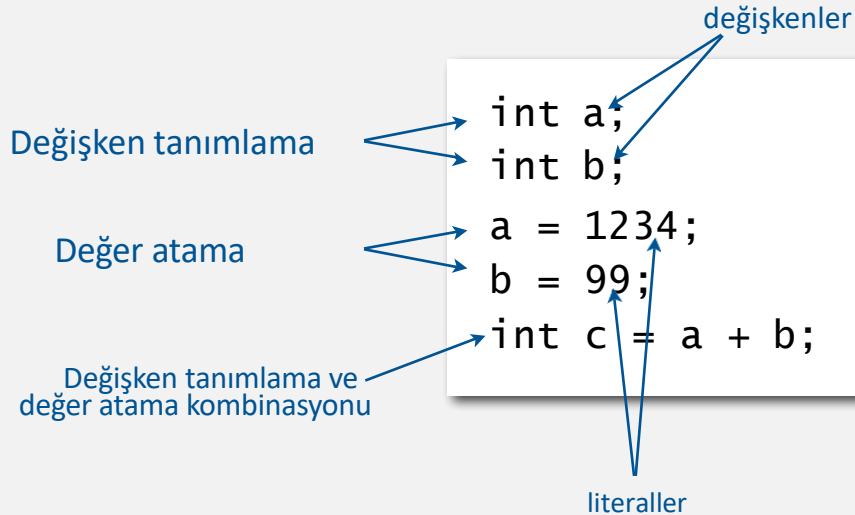
Temel Tanımlamalar

Değişken (variable) bir değere karşılık gelen referanstır.

Literal bir değerın programlama dili gösterimidir.

Değişken Tanımlama bir tip ile değeri ilişkilendirir.

Değer atama bir değer ile değişkeni ilişkilendirir.



Temel Ver Tiplerini Tanımlama ve İlk Atama Kuralları

```
int a = 0b101010; // binary
short b = 052;    // octal
byte c = 42;      // decimal
long d = 0x2A;    // hex
float e = 1.99E2F;
double f = 1.99;
long g = 5, h = c;
float i = g;
char j = 'A';
char k = '\u0041', l = '\101';
int s;
s = 77;
```



```
byte a;
byte b = a;
byte c = 128;
int d = 42L;
float e = 1.2;
char f = "a";
char g = 'AB';
boolean h = "true";
boolean i = 'false';
boolean j = 0;
boolean k = False;
```



Değişkenler, literaller, atama ve tanımlama. Örnek: değerler arası yer değiştirme

```
public class Exchange
{
    public static void main(String[] args)
    {
        int a = 1234;
        int b = 99;
        int t = a;
        a = b;
        b = t;
    }
}
```

Bu kod a ve b
değerlerini yer değiştirir

İzleme (trace) her komuttan sonraki değişken değerleri tablosudur.

	a	b	t
	<i>undeclared</i>	<i>undeclared</i>	<i>undeclared</i>
<code>int a = 1234;</code>	1234	<i>undeclared</i>	<i>undeclared</i>
<code>int b = 99;</code>	1234	99	<i>undeclared</i>
<code>int t = a;</code>	1234	99	1234
<code>a = b;</code>	99	99	1234
<code>b = t;</code>	99	1234	1234

Metinlerle işlem yapmak için veri tipi: String

String veri tipi

Değerler	Karakter dizileri
Tipik literaller	"Hello, " "1 " " * "
İşlem	birleştirme
operator	+

String işlemleri (concatenation)

İfade	değer
"Hi, " + "Bob"	"Hi, Bob"
"1" + " 2 " + "1"	"1 2 1"
"1234" + " " + " " + "99"	"1234 + 99"
"1234" + "99"	"123499"

Tipik Kullanım Alanı: Giriş/Çıkış İşlemleri

Önemli Not:

Karakterlerin yorumlanması içeriğe bağlıdır.

Ör 1: artı işareti

"1234" + " " + " " + "99"

↑ ↑
operatör operatör

↑
karakter

Ör 2: boşluklar

"1234" + " " + " " + "99"

white space white space

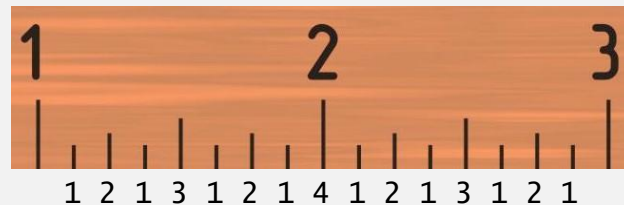
↙ ↘
↘ ↙

Boşluk karakteri

String işlemleri Örneği: Cetvelin alt bölümleri

```
public class Ruler
{
    public static void main(String[] args)
    {
        String ruler1 = "1";
        String ruler2 = ruler1 + " 2 " + ruler1;
        String ruler3 = ruler2 + " 3 " + ruler2;
        String ruler4 = ruler3 + " 4 " + ruler3;
        System.out.println(ruler4);
    }
}
```

all + ops are concatenation



% java Ruler

1 2 1 3 1 2 1 4 1 2 1 3 1 2 1

	ruler1	ruler2	ruler3	ruler4
	<i>undeclared</i>	<i>undeclared</i>	<i>undeclared</i>	<i>undeclared</i>
ruler1 = "1";	1	<i>undeclared</i>	<i>undeclared</i>	<i>undeclared</i>
ruler2 = ruler1 + " 2 " + ruler1;	1	1 2 1	<i>undeclared</i>	<i>undeclared</i>
ruler3 = ruler2 + " 3 " + ruler2;	1	1 2 1	1 2 1 3 1 2 1	<i>undeclared</i>
ruler4 = ruler3 + " 4 " + ruler3;				1 2 1 3 1 2 1 4 1 2 1 3 1 2 1

Giriş / Çıkış

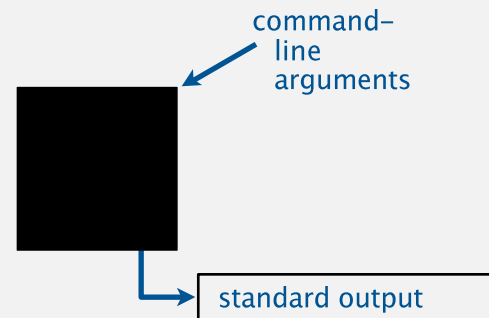
Programımıza işlenmek üzere veri sunmamız ve işlenen verilerin sonuçlarını alabilmemizi sağlarlar

Kullanıcılar metinlerle çalışmayı tercih ederler.

Programlar sayılarla daha etkili çalışırlar.

Çıkış

- `System.out.println()` verilen metni yazdıran metot.
- Java sayıları çıktı vermek için otomatik olarak metne çevirir.



Komut satırı Girdisi

- Çalıştırma sırasında programa verilen `args[0]`, `args[1]`, ... şeklindeki string girdilerdir.
- S. *integer* bir değeri komut satırı girdisi olarak nasıl alabiliriz?
- C. String ifadeleri tam sayıya çevirmek için kullanılan `Integer.parseInt()` sistem metodunu çağırırız.

Giriş / çıkış alıştıırma: değeri yer değıştirme

```
public class Exchange
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int t = a;
        a = b;
        b = t;
        System.out.println(a);
        System.out.println(b);
    }
}
```

Java çıktı işlemi için integer değeri otomatik olarak String e çevirir.

```
% java Exchange 5 2
2
5

% java Exchange 1234 99
99
1234
```

S. Bu program ne yapar?

A. Komut satırından iki değeri okur,daha sonra içeriklerini birbirini arasında değıştirir.

Tam sayı işlemleri için veri tipi: **int**

int veri tipi

Değerler	-2^{31} and $2^{31}-1$ arasındaki tamsayılar				
Tipik literaller	1234	99	0	1000000	
Operasyonlar	topla	çıkart	çarp	böl	mod
operatör	+	-	*	/	%

Önemli Not:

Yalnızca 2^{32} farklı int değer.

int operasyon örnekleri

ifade	değer	Açıklama
5 + 3	8	
5 - 3	2	
5 * 3	15	
5 / 3	1	Tam kısmını alır
5 % 3	2	Mod alır
1 / 0		runtime error

Öncelik

ifade	değer	Açıklama
3 * 5 - 2	13	* öncelik sahibi
3 + 5 / 2	5	/ öncelik sahibi
3 - 5 - 2	-4	Soldan sıralı
(3 - 5) - 2	-4	Tercih edilen

Tipik kullanım: Matematik hesaplamaları

Int ve String ifadeler arasında tip dönüşümü ile işlemler

```
public class IntOps
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int sum = a + b;
        int prod = a * b;
        int quot = a / b;
        int rem = a % b;
        System.out.println(a + " + " + b + " = " + sum);
        System.out.println(a + " * " + b + " = " + prod);
        System.out.println(a + " / " + b + " = " + quot);
        System.out.println(a + " % " + b + " = " + rem);
    }
}
```

↑
Java String birleştirme işlemi için integer değeri otomatik olarak String e çevirir.

```
% java IntOps 5 2
```

```
5 + 2 = 7
```

```
5 * 2 = 10
```

```
5 / 2 = 2
```

```
5 % 2 = 1
```

```
% java IntOps 1234 99
```

```
1234 + 99 = 1333
```

```
1234 * 99 = 122166
```

```
1234 / 99 = 12
```

```
1234 % 99 = 46
```

Ondalıklı sayılarla işlem yapabilmek için: double

double veri tipi

değerler	real numbers				
tipik literaller	3.14159	2.0	1.4142135623730951	6.022e23	
operasyonlar	topla	çıkar	çarp	böl	mod
operatörler	+	-	*	/	%

6.022 x 10²³

Tipik double değerler yaklaşık değerlerdir

Örnek:

π için double değer yoktur.

$2^{1/2}$ için double değer yoktur.

$1/3$ için double değer yoktur.

double operasyon örnekleri

ifade	değer
3.141 + .03	3.171
3.141 - .03	3.111
6.02e23/2	3.01e23
5.0 / 3.0	1.6666666666666667
10.0 % 3.141	0.577
Math.sqrt(2.0)	1.4142135623730951

Özel değerler

ifade	değer
1.0 / 0.0	Infinity
Math.sqrt(-1.0)	NaN

"not a number"

Ondalıklı sayılar örneği: ikinci derece denklemler

Cebir örneği: $x^2 + bx + c$ için kökler $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

```
public class Quadratic
{
    public static void main(String[] args)
    {

        // Parse coefficients from command-line.
        double b = Double.parseDouble(args[0]);
        double c = Double.parseDouble(args[1]);

        // Calculate roots of x*x + b*x + c.
        double discriminant = b*b - 4.0*a*c;
        double d = Math.sqrt(discriminant);
        double root1 = (-b + d) / 2.0;
        double root2 = (-b - d) / 2.0;

        // Print them out.
        System.out.println(root1);
        System.out.println(root2);

    }
}
```

```
% java Quadratic -3.0 2.0
2.0
1.0
x2 - 3x + 2

% java Quadratic -1.0 -1.0
1.618033988749895
-0.6180339887498949
x2 - x - 1

% java Quadratic 1.0 1.0
NaN
NaN
x2 + x + 1

% java Quadratic 1.0 hello
java.lang.NumberFormatException: hello

% java Quadratic 1.0
java.lang.ArrayIndexOutOfBoundsException
```

İki argüman gerekli.

(Hayatın gerçekleri: Tüm hatalar anlaşılır değildir.)

true/false verilerde işlemler: boolean

boolean veri tipi

<i>değerler</i>	true	false	
<i>literaller</i>	true	false	
<i>operasyonlar</i>	and	or	not
<i>operatörler</i>	&&		!

Doğruluk tablosu tanımları

a	!a	a	b	a && b	a b
true	false	false	false	false	false
false	true	false	true	false	true
		true	false	false	true
		true	true	true	true

S. a XOR b?

C. (!a && b) || (a && !b)

İspat

a	b	!a && b	a && !b	(!a && b) (a && !b)
false	false	false	false	false
false	true	true	false	true
true	false	false	true	true
true	true	false	false	false

Boolean ile hesaplama örneği: artık yıl kontrolü

Q. Girilen yıl artık yıl mıdır?

A. Eğer (i) 400 e bölünebilen veya (ii) 4 e bölünebilen fakat 100 e bölünmeyen

```
public class LeapYear
{
    public static void main(String[] args)
    {
        int year = Integer.parseInt(args[0]);
        boolean isLeapYear;

        // divisible by 4 but not 100
        isLeapYear = (year % 4 == 0) && (year % 100 != 0);

        // or divisible by 400
        isLeapYear = isLeapYear || (year % 400 == 0);

        System.out.println(isLeapYear);
    }
}
```

```
% java LeapYear 2016
true
```

```
% java LeapYear 1993
false
```

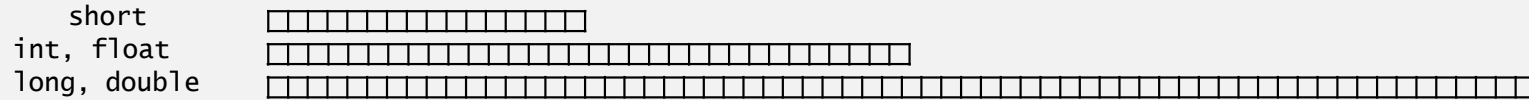
```
% java LeapYear 1900
false
```

```
% java LeapYear 2000
true
```

Tip dönüşümlerinin hikayesi

Neden farklı nümerik tipler?

- Bellek kullanımı ve ihtiyaç duyulan sayı aralığı konusundaki tercihler.
- Bellek kullanımı ve hassasiyet ihtiyacı konusundaki tercihler.



Bir dönüşümün **imkansız** olduğu durumlar..

- Ör: **(short) 70000**.
- Short değerler -2^{15} and $2^{15} - 1 = 32767$.

Java'nın Math kütüphanesinden örnekler

- Küçük veri tipleri büyük veri tiplerine otomatik olarak dönüştürülür.
- Büyük veri tipleri küçük veri tiplerine dönüştürme yapmadan atanamaz.
- Dönüştürme **(yeni tip) değişken veya ifade** şeklinde yapılabilir.
- Büyük tipler küçük tiplere dönüştürülürken taşma meydana gelebilir.
- **int** tipinden küçük tiplerin aritmetik işleminin sonucu **int** tir. Sonuç en büyük katılımcının tipidir.

```
byte a = 127, b = 5;
❌ byte c = a+b;           // compilation fails
    int
✅ int d = a + b;          // d is 132
⚠ byte e = (byte) (a+b);   // e is -124 (type overflow, because 127 is the max byte value)
⚠ int f = a/b;             // f is 25 (a/b is 25 because it is an int)
⚠ float g = a/b;           // g is 25.0F (result of the a/b can be implicitly or
⚠ float h = (float) (a/b); // h is 25.0F explicitly casted to float, but a/b is still 25)
✅ float i = (float)a/b;    // i is 25.4F (when either a or b
✅ float j = a/(float)b;    // j is 25.4F is float the a/b becomes float)
⚠ b = (byte) (b+1);        // explicit casting is required, because b+1 is an int
✅ b++;                    // no casting is required for ++ and -- operators
char x = 'x';
✅ char y = ++x;           // arithmetic operations work with character codes
```

Java'nın Math kütüphanesinden örnekler

public class Math	
double abs(double a)	Mutlak değer
double max(double a, double b)	Max a ve b
double min(double a, double b)	Min a ve b
double sin(double theta)	Sin fonksiyonu
double cos(double theta)	cos fonksiyonu
double tan(double theta)	tan fonksiyonu
double exp(double a)	(e^a)
double log(double a)	doğal log ($\log_e a$, or $\ln a$)
double pow(double a, double b)	(a^b)
long round(double a)	En yakın tamsayıya yuvarla
double random()	[0. 1) arası random sayı
double sqrt(double a)	karekök a
double E	e (constant)
double PI	π (constant)

int, long, ve float için de tanımlanabilir

asin(), acos(), and atan() ters fonksiyonları da mevcuttur.

Radian olarak alır. toDegrees() ve toRadians() dönüşüm de kullanılabilir



Artık hesap makinesini bırakabilirsiniz. (lütfen!).

Karşılaştırma Operatörleri

Temel veri tipleri arasında **karşılaştırma** yapmak için kullanılır.

- Operand: Aynı veri tipindeki iki değer.
- Sonuç: Boolean bir değer.

<i>operatör</i>	<i>Anlamı</i>	true	false
==	Eşit	2 == 2	2 == 3
!=	Eşit değil	3 != 2	2 != 2
<	Küçük	2 < 13	2 < 2
<=	Büyük eşit	2 <= 2	3 <= 2
>	Büyük	13 > 2	2 < 13
>=	Büyük eşit	3 >= 2	2 >= 3

Örnekler

non-negative discriminant?

`( b*b - 4.0*a*c ) >= 0.0`

Yüzyılın başlangıcı?

`( year % 100 ) == 0`

Doğru ay değeri?

`( month >= 1 ) && ( month <= 12 )`

Tipik double değerler
yaklaşık değer
olduğundan ==
karşılaştırmalarında
dikkatli olun

Karşılaştırma Operatörleri

Operators	Precedence
postfix increment and decrement	++ --
prefix increment and decrement, and unary	++ -- + - ~ !
multiplicative	* / %
additive	+ -
bit shift	<< >> >>>
relational	< > <= >= instanceof
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	
logical AND	&&
logical OR	
ternary	? :
assignment	= += -= *= /= %= &= ^= = <<= >>= >>>=

Atama ve Aritmetik Operatörler



Atama ve Aritmetik

= + - * / %

Birleşik atamalar

+= -= *= /= %=

Parantez ile işlem sırası değişebilir.

()

Artırma ve Azaltma işlemleri ön-ek ve son-ek şeklinde yazılabilir.

++ --

```
int a = 1;    // assignment (a is 1)
int b = a+4;  // addition (b is 5)
int c = b-2;  // subtraction (c is 3)
int d = c*b;  // multiplication (d is 15)
int e = d/c;  // division (e is 5)
int f = d%6;  // modulus (f is 3)
```

```
int a = 1, b = 3;
a += b; // equivalent of a=a+b (a is 4)
a -= 2; // equivalent of a=a-2 (a is 2)
a *= b; // equivalent of a=a*b (a is 6)
a /= 2; // equivalent of a=a/2 (a is 3)
a %= a; // equivalent of a=a%a (a is 0)
```

```
int a = 2, b = 3;
int c = b-a*b;    // (c is -3)
int d = (b-a)*b;  // (c is 3)
```

```
int a = 1, b = 0;
a++;    // increment (a is 2)
++a;    // increment (a is 3)
a--;    // decrement (a is 2)
--a;    // decrement (a is 1)
b = a++; // increment postfix (b is 1, a is 2)
b = ++a; // increment prefix  (b is 3, a is 3)
b = a--; // decrement postfix (b is 3, a is 2)
b = --a; // decrement prefix  (b is 1, a is 1)
```

Tip kontrol

Değişken tipleri daima operasyonlardaki tanımlarla eşleşmelidir.

Java compiler sizin dostunuzdur: kodunuzdaki tip hatalarını kontrol eder.

```
public class BadCode
{
    public static void main(String[] args)
    {
        String s = "123" * 2;
    }
}
```

```
% javac BadCode.java
BadCode.java:5: operator * cannot be applied to java.lang.String,int
        String s = "123" * 2;
                        ^
1 error
```

Uygun olduğunda, sık sık bir tipteki değeri başka bir tipe, tip uyumunu sağlamak için dönüştürürüz

Temel veri tiplerinde tip dönüşümü

Tip dönüşümleri programlamanın doğasında olan işlemlerdir.

Otomatik

- "+" için sayıları stringe dönüştürme.
- Hassasiyet kaybı yok ise sayısal tiplerin dönüşümü.

Fonksiyon parametrelerinde açıkça tanımlı durumlar

<i>ifade</i>	<i>tip</i>	<i>değer</i>
"x: " + 99	String	"x: 99"
11 * 0.25	double	2.75

Integer.parseInt("123")	int	123
Math.round(2.71828)	long	3

(int) 2.71828	int	2
(int) Math.round(2.71828)	int	3
11 * (int) 0.25	int	0



Verinizin tipine dikkat edin!

← Tip dönüşümleri mantıksız gelebilir ancak uygulamada oldukça pratik çözümler sunar

Tip dönüşümü Quiz

S. Aşağıdaki ifadelerin her birinin tip ve sonuçlarını belirtiniz.

a. $(7 / 2) * 2.0$

b. $(7 / 2.0) * 2$

c. $"2" + 2$

d. $2.0 + "2"$

Tip dönüşümü Quiz

S. Aşağıdaki ifadelerin her birinin tip ve sonuçlarını belirtiniz.

a. $(7 / 2) * 2.0$ 6.0, `double` (7/2 is 3, an `int`)

b. $(7 / 2.0) * 2$ 7.0, `double`

c. `"2" + 2` 22, `String`

d. `2.0 + "2"` 2.02, `String`

Tip dönüşümünün kullanışlı olduğu durumlar: Random integer lar

`Math.Random()` sistem metodu $[0,1)$ aralığında rastgele sayılar üretir.

Problem: N sayısı girilsin, 0 ile $N-1$ arasında bir random tam sayı üretin.

```
public class RandomInt
{
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        double r = Math.random();
        int t = (int) (r * N);
        System.out.println(t);
    }
}
```

String to int (system method)

double to int (cast)

int to double (automatic)

```
% java RandomInt 6
3
```

```
% java RandomInt 6
0
```

```
% java RandomInt 10000
3184
```

Binary Sayı Gösterimi

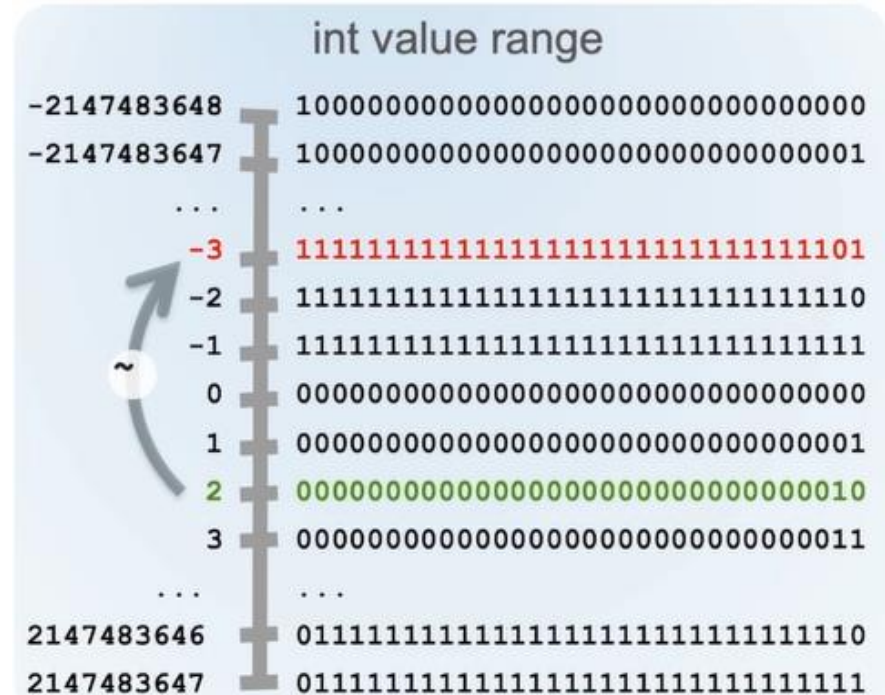
Tüm Java nümerik tipleri işaretlidir (pozitif ve negatif)

- Java bir tamsayının büyüklüğünü ifade eden sayı için 2.tümleyen uygulamasını kullanır.
- Ör: 1 bytelık 0 değeri 00000000 şeklindeki bit dizisi olarak gösterilir.
- İşaret değişikliği (pozitif veya negatif) tüm bitlerin invert edilmesi ve üzerine 1 eklenmesiyle yapılır.
- Ör: 1 bytelık 1 değeri 00000001 ve -1 değeri 11111111 şeklinde gösterilir.

Bitsel tümleyen operatörü (**~a**) tüm bitleri bir sayıya çevirir:

Sonucu mirrored ikili karşılığıdır: $-(a+1)$

```
int a = 2; //  
int b = ~a; // b is -3
```



Bitset Operatörler

<i>Operatör</i>	<i>Anlamı</i>
&	AND
	OR
^	XOR
<<	Signed Left Shift
>>	Signed Right Shift
>>>	Unigned Right Shift

```
byte a = 5;           // 00000101
byte b = 3;           // 00000011
byte c = (byte) (a & b); // 00000001 (c is 1)
byte d = (byte) (a | b); // 00000111 (d is 7)
byte e = (byte) (a ^ b); // 00000110 (e is 6)
```

[illegible]

Kısa Devre Değerlendirmesi

`&& ||` (short-circuit evaluation)

`& | ^` (full evaluation)

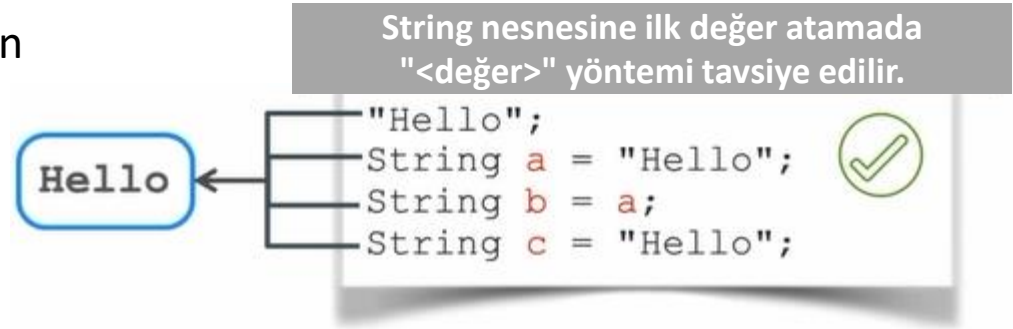
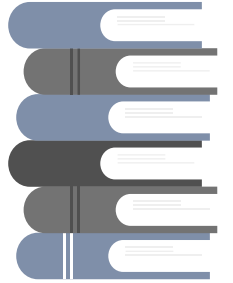
```
true && evaluated
false && not evaluated
false & evaluated
false || evaluated
true || not evaluated
true | evaluated
true ^ evaluated
false ^ evaluated
```

```
int a = 3, b = 2;
boolean c = false;
c = (a > b && ++b == 3); // c is true, b is 3
c = (a > b && ++b == 3); // c is false, b is 3
c = (a > b || ++b == 3); // c is false, b is 4
c = (a < b || ++b == 3); // c is true, b is 4
c = (a < b | ++b == 3); // c is true, b is 5
```

Java'da String

`java.lang.String` sınıfı bir karakter dizisi sunar.

- `String` bir sınıftır (primitif değildir). Örnekleri birer karakter dizisidir.
- Diğer Java nesnelerinde olduğu gibi, `String` nesnesi de **new** keywordü kullanılarak oluşturulabilir.
- `String` çift tırnak içerisinde yazılan bir metinle örnek oluşturan tek nesnedir ve bu kullanım tavsiye edilen yöntemdir.
- JVM her `String` literalı için bellekte tek kopya tutarak bellek optimizasyonu yapar.(Bu işleme *interning* denir)
- `intern()` metodu bir `String` literalinin intern edilmiş kopyasına referans döndürür.



String Operasyonları

String nesneleri değiştirilemezler.

- Bir string nesneye ilk değer ataması yapıldıktan sonra değiştirilemezler.
- `trim()`, `concat()`, `toLowerCase()`, `toUpperCase()` gibi metotlar yeni bir String döndürecektir, orijinal String'i değiştirmezler.
- Bu işlem string referansını, *farklı bir string nesnesini referans ederek yeniden atama* ile mümkündür.
- Kolaylık olması açısından `+` operatörü `concat()` yerine kullanılır, fakat `+` işaretinin bir aritmetik operatör olduğunu unutmayın.

```
String s = "";  
s=1+1+"u";    // s is 2u  
s="u"+1+1;    // s is u11  
s="u"+(1+1);  // s is u2
```



String İndeksleme

String, **integer ile indekslenmiş** bir karakter dizisi içerir.

- İndeks 0'dan başlar.
- substring alırken **başlangıç indisi** dahil, **bitiş indisi** hariçtir.
- Bir substring bulunamazsa, indexOf metodu -1 değeri döndürür.
- indexOf ve lastIndexOf operasyonlarının ikisi de **overloaded** metotlardır. Bir karakter veya String değeri parametre olarak alabilirler.
- Son karakter indisine uzunluk-1 olarak erişilebilir.

0	1	2	3	4	5	6	7	8	9
H	e	l	l	o	W	o	r	l	d

```
String a = "HelloWorld";  
String b = a.substring(0,5); // b is "Hello"  
int c = a.indexOf('o');      // c is 4  
int d = a.indexOf('o',5);    // d is 6  
int e = a.lastIndexOf('l');  // e is 8  
int f = a.indexOf('a');      // f is -1  
char g = a.charAt(0);        // g is H  
int h = a.length();          // h is 10  
char i = a.charAt(10);
```

⊗ throw StringIndexOutOfBoundsException

Temel Veri Tipleri için Wrapper Sınıflar

Wrapper sınıflar temel veri tiplerine nesne-tabanlı kabiliyetler kazandırır.

- Bir Wrapper sınıfı, her temel veri tipi için sağlanan temel değerleri tutabilir.
- Bir temel tip veya string'den wrapper nesne oluşturmak için `valueOf()` metodu kullanabilirsiniz
- Wrapper dışına temel tip değerini çıkarmak için `xxxValue()` metodlarını kullanabilirsiniz.
- Yukarıdaki iki işlemi formal yolla yapmak yerine **auto-boxing** ve **auto-unboxing** diye adlandırılan doğrudan atama yolunu da tercih edebilirsiniz.
- String için wrapper oluşturmak veya temel yapıya dönüştürmek için `parseXXX()` metotları kullanılır.
- Bir temel tipi string e dönüştürmek için `String.valueOf()` metodu kullanılır.
- Wrapper sınıflar, **min ve max değerleri** gibi tüm tipler için sabitler sunar.

Primitive Wrapper

byte Byte

short Short

int Integer

long Long

float Float

double Double

char Character

boolean Boolean

```
int a = 42;
Integer b = Integer.valueOf(a);
int c = b.intValue();
b = a;
c = b;
String d = "12.25";
Float e = Float.valueOf(d);
float f = d.parseFloat();
String b = String.valueOf(a);
Short.MIN_VALUE;
Short.MAX_VALUE;
```

Temel veri tipleri

Koşul ifadeleri

Diziler ve döngüler

Metotlar

İçerik: programlamanın temel taşları

Yazmak isteyebileceğin herhangi bir program

Nesneler

Fonksiyonlar ve Modüller

Grafik, Ses ve Görüntü I/O

Diziler ve Döngüler

Koşullar

Math

text I/O

Temel veri tipleri

Atama ifadeleri



Bu Bölüm:
sonsuzluk ve ötesi

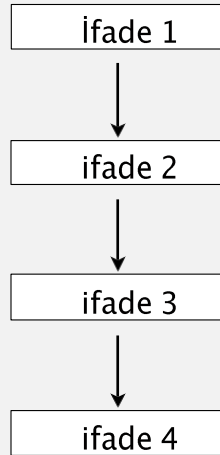


Önceki Bölüm: Bir hesap
makinesine eşdeğerdir

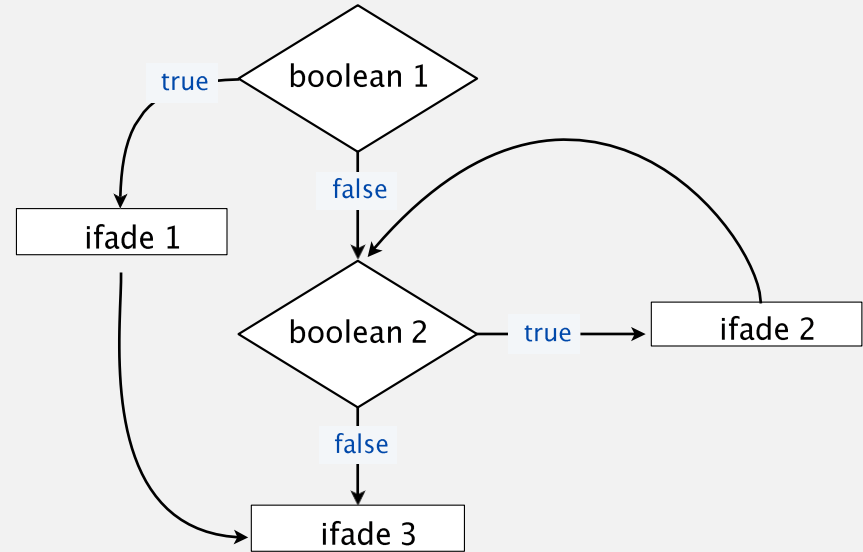
Koşullar ve Döngüler

Kontrol Akışı

- Bir programda fiilen yürütülen komutlar dizisi.
- **Koşullar ve döngüler** kontrol akışının bir koreografisini yapmamızı sağlar.



Düz hat kontrol akışı
[önceki bölümde]



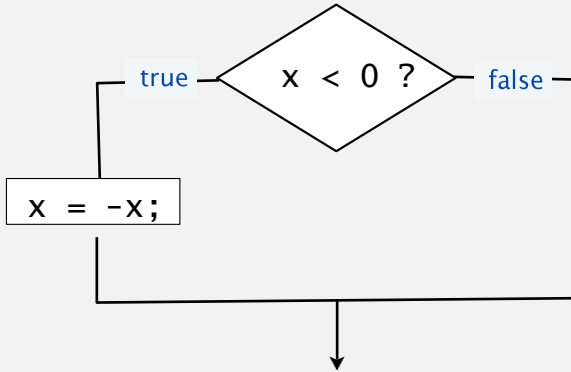
Koşullar ve döngülerle kontrol akışı
[Bu bölüm]

if-else deyimi

Belirli değişkenlerin değerlerine bağlı olarak belirli komutlar çalıştırır.

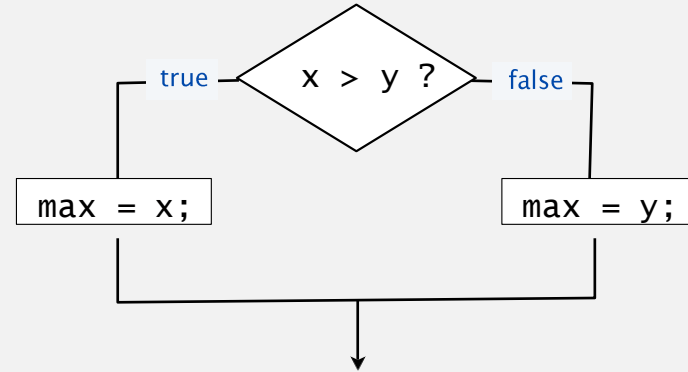
- Boolean ifadeleri kullanır.
- `true` ise komut çalıştırılır.
- `else` : `false` ise farklı bir komutu çalıştırır.

Örnek: `if (x < 0) x = -x;`



x değerini mutlak değeriyle değiştirir.

Örnek: `if (x > y) max = x;`
`else max = y;`



x ve y nin maksimum olanını belirler

if ifadesi için örnek: yazı-tura oyunu

```
public class Flip
{
    public static void main(String[] args)
    {
        if (Math.random() < 0.5)
            System.out.println("Yazı");
        else
            System.out.println("Tura");
    }
}
```

```
% java Flip
Yazı
```

```
% java Flip
Yazı
```

```
% java Flip
Tura
```

```
% java Flip
Yazı
```



if kullanımı örnek: 2-değer sıralama

S. Bu program ne yapar?

```
public class TwoSort
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        if (b < a)
        {
            int t = a;
            a = b;
            b = t;
        }
        System.out.println(a);
        System.out.println(b);
    }
}
```

if – else deyiminin alternatifi
← bir ifadeler topluluğu
şeklinde kullanılabilir.

```
% java TwoSort 1234 99
99
1234
```

```
% java TwoSort 99 1234
99
1234
```

C. Komut satırından iki integer değer okur ve onları sıralı şekilde yazdırır.

if kullanımı örnek

S. Aşağıdaki programa a, b ve c sayılarını sıralayan bir kod ekleyiniz.

```
public class ThreeSort
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int c = Integer.parseInt(args[2]);

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
    }
}
```

```
% java ThreeSort 1234 99 1
1
99
1234
```

```
% java ThreeSort 99 1 1234
1
99
1234
```

if kullanımı örnek

S. Aşağıdaki programa a, b ve c sayılarını sıralayan bir kod ekleyiniz.

C.

```
public class ThreeSort
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int c = Integer.parseInt(args[2]);
        if (b < a)
        { int t = a; a = b; b = t; } ← a yı b den küçük
        if (c < a)                               yapar
        { int t = a; a = c; c = t; } ← a yı b den ve c
        if (c < b)                               den küçük yapar
        { int t = b; b = c; c = t; } ← b yi c den küçük
        System.out.println(a);                  yapar
        System.out.println(b);
        System.out.println(c);
    }
}
```

```
% java ThreeSort 1234 99 1
1
99
1234
```

```
% java ThreeSort 99 1 1234
1
99
1234
```

if kullanımı örnek: hata kontrolü

```
public class IntOps
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int sum = a + b;
        int prod = a * b;
        System.out.println(a + " + " + b + " = " + sum);
        System.out.println(a + " * " + b + " = " + prod);
        if (b == 0) System.out.println("Division by zero");
        else        System.out.println(a + " / " + b + " = " + a / b);
        if (b == 0) System.out.println("Division by zero");
        else        System.out.println(a + " % " + b + " = " + a % b);
    }
}
```

```
% java IntOps 5 2
5 + 2 = 7
5 * 2 = 10
5 / 2 = 2
5 % 2 = 1
```

```
% java IntOps 5 0
5 + 0 = 5
5 * 0 = 0
Division by zero
Division by zero
```

İyi programlama önerisi: Runtime hataların kontrolünde ve önlenmesinde koşul ifadelerini kullanın.

Ternary (Üçlü) Operatör

Koşullu atamaları düzenlemek için kullanılır.

Ternary operatörü **?**; Yalnızca bir koşula göre bir atamanız gerekiyorsa, if/else yapısının yerine kullanılır.

Boolean ifade **true** değer üretirse, **' ? '** den sonraki değer atanır.

Boolean ifade **false** değer üretirse, **' : '** den sonraki değer atanır.

```
<variable> = (<boolean expression>) ? <value one> : <value two>;
```

```
int a = 2, b = 3;  
int c = (a >= b) ? a : b; // c is 3
```



Bu yapılar aynı sonucu üretir

```
int a = 2, b = 3;  
int c = 0;  
if (a >= b) {  
    c = a;  
} else {  
    c = b;  
}
```



Not: Ternary operatörü koşul atamalarını sadeleştirmek için kullanılır. if/else ifadesinin yerine kullanımı kodunuzun okunabilirliğini olumsuz etkiler.

```
int a = 2, b = 3;  
int c = (a >= b) ? a : (--b == a) ? a : b; // c is 2
```



switch ile Akış Kontrolü

- Switch **expression** şu tiplerden biri olmalıdır: **byte, short, int, char, String, enum**.
- Case **label expression** tipiyle aynı olmalıdır.
- Çalışma akışı expression değeriyle eşleşen case bloğuna geçer.
- Çalışma akışı switch bloğunun sonuna gelene kadar veya **break** ifadesini görene kadar devam eder.
- switch expression case değerlerinden hiçbiri ile eşleşmezse **default** bloğu çalıştırılır. Sonda olmak zorunda değildir ve kullanımı opsiyoneldir.

```
switch (<expression>) {  
    case <label>:  
        <case logic>  
    case <label>:  
        <case logic>  
        break;  
    default:  
        <case logic>  
}
```

Örnek durumlar:

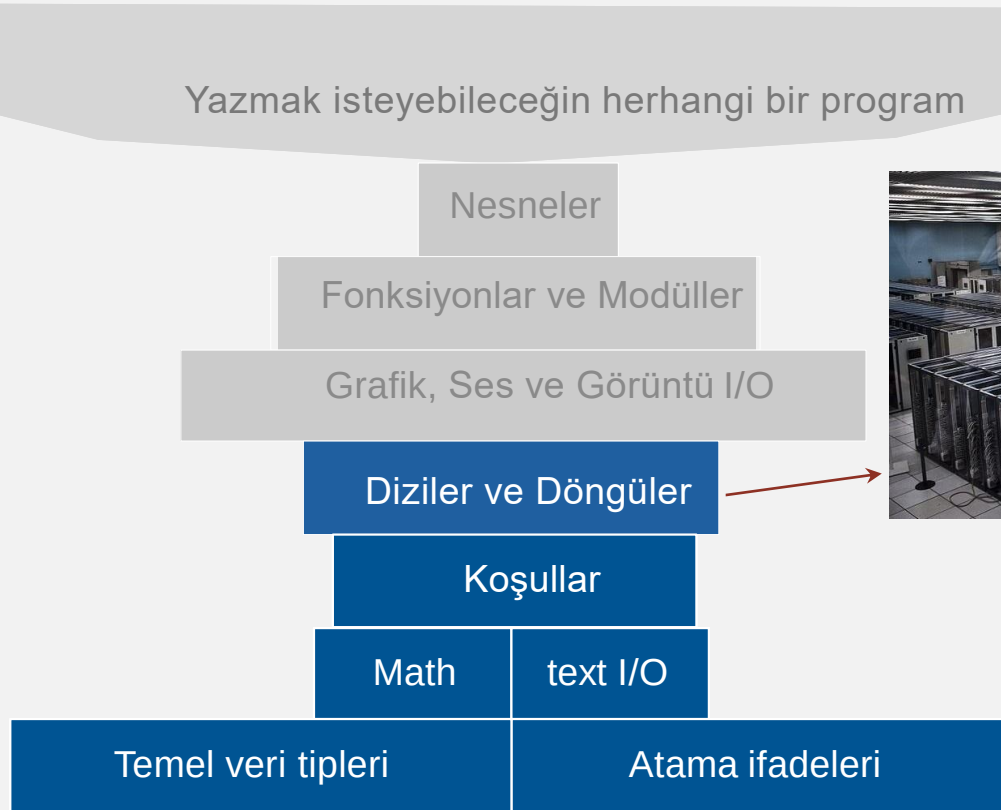
- Special, fiyatı 1 artır.
- New, fiyatı 2 artır.
- Discounted, fiyatı 4 azalt.
- Expired, fiyatı 0 yap.
- Diğer durumlar için fiyatı 3 yap.

status 'N' (New) olduğunda Switch
içerisindeki çalışma güzergahı

```
char status = 'N';  
double price = 10;  
switch (status) {  
    case 'S':  
        price += 1; // not executed  
    case 'N':  
        price += 2; // price is 12  
    case 'D':  
        price -= 4; // price is 8  
        break;  
    case 'E':  
        price = 0; // not executed  
        break;  
    default:  
        price = 3; // not executed  
}  
//the rest of the program logic
```

Temel veri tipleri
Koşul ifadeleri
Diziler ve Döngüler
Metotlar

Programlama için temel yapısal bloklar



Büyük miktarda veriyi saklama ve işleme yeteneği

Diziler

Bir dizi, `int` tipinde bir sayı ile indekslenmiş, aynı veri tipindeki elemanların sabit uzunluklu bir koleksiyonudur.

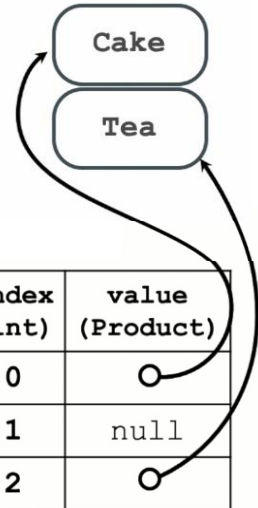
- **Dizi nesnesini tanımlamak** – Dizide saklanacak verilerin tipini belirle.
- **Dizi nesnesini oluşturmak** – Dizideki uzunluğunu (eleman sayısı) belirle.
 - Dizi bir kere oluşturulduktan sonra uzunluğu değiştirilemez.
 - Nesne dizileri `null` değerle doldurulur.
 - Temel veri tipi dizileri `0` ile doldurulur(boolean için `false`)
- **Dizi içeriğine ilk değer atama** – Dizi pozisyonlarına atama herhangi bir sıra ile olabilir.
 - İndis 0'dan başlar.
 - Son indis `array.length-1` dir.
- **Dizi elemanlarına indis kullanarak erişilir.**
 - Dizi elemanları ile işlem yapmak için diziden dışarı taşınmasına gerek yoktur.
 - İndis değerleri dışında bir erişim talebi olursa `ArrayIndexOutOfBoundsException` hatası verir.

```
int[] primes;  
// int primes[];  
primes = new int[4];  
primes[1] = 3;  
primes[2] = 7;  
primes[0] = primes[1]-1;  
primes[4];
```

index (int)	value (int)
0	2
1	3
2	7
3	0

```
Product[] products;  
// Product products[];  
products = new Product[3];  
products[0] = new Food("Cake");  
products[2] = new Drink("Tea");  
products[2].setPrice(1.99);  
products[3];
```

index (int)	value (Product)
0	
1	null
2	



Dizilerde Birleştirilmiş Tanımlama, Oluşturma ve İlk Değer Atama

Dizi nesneleri için tanımlama, oluşturma ve ilk değer atama aynı anda yapılabilir.

- Dizi nesnesinin birleştirilmiş tanımlama ve oluşturulması:

```
int[] primes = new int[3];  
primes[0] = 2;  
primes[1] = 3;  
primes[2] = 5;
```

```
Product[] products = new Product[3];  
products[0] = new Food("Cake");  
products[1] = new Drink("Tea");  
products[2] = new Food("Cookie");
```

- Dizi nesnesinin oluşturulması ve ilk değer atamasının birleştirilmesi:

```
int[] primes;  
primes = new int[]{2,3,5};
```

```
Product[] products;  
products = new Product[]{new Food("Cake"),  
                           new Drink("Tea"),  
                           new Food("Cookie")};
```

- Dizi nesnesinin tanımlanması ve oluşturulması ile ilk değer atamasının birleştirilmesi.

```
int[] primes = {2,3,5};
```

```
Product[] products = {new Food("Cake"),  
                       new Drink("Tea"),  
                       new Food("Cookie")};
```

Çok Boyutlu Diziler

Java dizileri çok boyutlu olabilir:

- Normal veya kısa yoldan başlatma yapılabilir.
- Her boyuta ayrı indisle erişim sağlanır.
- Kare olmayan bir şekil içerebilir.
- İki den fazla boyutta tanımlanabilir.

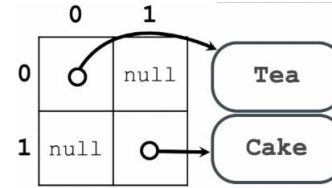
```
int[][] matrix = new int[2][3];  
matrix[0][1] = 5;  
matrix[1][2] = 7;
```

	0	1	2
0	0	5	0
1	0	0	7

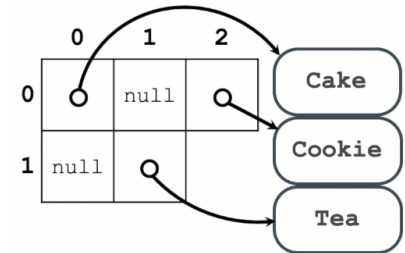
```
int[][] matrix = {{4,1},{2,0,5}};
```

	0	1	2
0	4	1	
1	2	0	5

```
Product[][] products = new Product[2][2];  
matrix[1][1] = new Food("Cake");  
matrix[0][0] = new Drink("Tea");
```



```
Product[][] products =  
    {{new Food("Cake"), null, new Food("Cookie")},  
     {null, new Drink("Tea")}};
```



Dizi İçeriğini Kopyalama

Dizi içeriği bir başka diziye kopyalanabilir:

- Java dizileri sabit uzunluktadır.(Yeniden boyutlandırılmaz)
- Yeniden boyutlandırma, farklı boyutta yeni bir dizi oluşturularak, dizinin içeriğinin tamamı veya bir kısmı kopyalanarak gerçekleştirilebilir:

```
System.arraycopy(<source array>, <source position>,  
                 <destination array>, <destination position>,  
                 <length of content to copy from source>);
```

- veya:

```
<new array> = Arrays.copyOf(<source array>, <new array length>);
```

```
<new array> = Arrays.copyOfRange(<source array>, <start position>, <end position>);
```

index int	value char
0	a
1	c
2	m
3	e

index int	value char
0	a
1	c
2	m
3	e
4	

```
char[] a1 = {'a','c','m','e'};  
char[] a2 = {'t','o','','',''};  
System.arraycopy(a1, 2, a2, 3, 2);
```

```
char[] b1 = {'a','c','m','e'};  
char[] b2 = Arrays.copyOf(b1,5);
```

index int	value char
0	a
1	c
2	m
3	e

index int	value char
0	t
1	o
2	
3	m
4	e

Arrays Sınıfı

java.util.Arrays sınıfı dizileri kullanmak için kullanışlı metotlar sunar. Şöyle ki:

- Diziye bir değerle doldurmak,
- Dizide arama yapmak,
- İçerik karşılaştırma,
- Dizi içeriğini sıralama:

- Comparable (dizi içindeki nesneler tarafından kullanıldığı gibi)
- Comparator interface

```
String[] values = new String[5];  
Arrays.fill(values, 2, 4, "aaa");  
int x = Arrays.binarySearch(values, "aaa");
```

0	null
1	null
2	aaa
3	aaa
4	null

```
String[] names1 = {"Mary", "Ann", "Jane", "Tom"};  
String[] names2 = {"Mary", "Ann", "John", "Tom"};  
boolean isTheSame = Arrays.equals(names1, names2);  
Arrays.sort(names2);  
Arrays.sort(names2, new LengthCompare());
```

```
public class LengthCompare implements Comparator<String> {  
    public int compare(String s1, String s2) {  
        if (s1.length() > s2.length()) { return 1; }  
        if (s1.length() < s2.length()) { return -1; }  
        return 0;  
    }  
}
```

0	Mary
1	Ann
2	Jane
3	Tom

0	Mary	0	Ann	0	Ann
1	Ann	1	John	1	Tom
2	John	2	Mary	2	John
3	Tom	3	Tom	3	Mary

Dizilerle programlama: tipik örnekler

Sistem dizisindeki komut-satırı argümanlarına erişim

```
int stake = Integer.parseInt(args[0]);  
int goal = Integer.parseInt(args[1]); int  
trials = Integer.parseInt(args[2]);
```

N adet random değerle bir dizi oluşturma

```
double[] a = new double[N];  
for (int i = 0; i < N; i++)  
    a[i] = Math.random();
```

Dizi değerlerinin ortalamasını hesaplama

```
double sum = 0.0;  
for (int i = 0; i < N; i++)  
    sum += a[i];  
double average = sum / N;
```

***N tüm kodlar için a nın ve b nin uzunluğudur.

Farklı bir diziye aktarım

```
double[] b = new double[N];  
for (int i = 0; i < N; i++)  
    b[i] = a[i];
```

Her satırda bir tane olacak şekilde dizi elemanlarını yazdır

```
for (int i = 0; i < N; i++)  
    System.out.println(a[i]);
```

Dizi değerlerinin maksimumunu bulma

```
double max = a[0];  
for (int i = 1; i < N; i++)  
    if (a[i] > max) max = a[i];
```

S. Aşağıdaki kodun çıktısı nedir?

```
public class PQarray1
{
    public static void main(String[] args)
    {
        int[] a = new int[6];
        int[] b = new int[a.length];

        b = a;
        for (int i = 1; i < b.length; i++)
            b[i] = i;

        for (int i = 0; i < a.length; i++)
            System.out.print(a[i] + " ");
        System.out.println();

        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
        System.out.println();
    }
}
```

Mini sınav: Diziler

S. Aşağıdaki kodun çıktısı nedir?

```
public class PQarray1
{
    public static void main(String[] args)
    {
        int[] a = new int[6];
        int[] b = new int[a.length];

        b = a;
        for (int i = 1; i < b.length; i++)
            b[i] = i;

        for (int i = 0; i < a.length; i++)
            System.out.print(a[i] + " ");
        System.out.println();

        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
        System.out.println();
    }
}
```

Bu ifadeden sonra a ve b aynı diziyi gösterir.

C.

```
% java PQ4_1
0 1 2 3 4 5
0 1 2 3 4 5
```

Dizilerle programlama: tipik hatalar

Array index out of bounds

```
double[] a = new double[10];  
for (int i = 1; i <= 10; i++)  
    a[i] = Math.random();
```

a[10] yok (ve a[0] kullanılmadı)



Uninitialized array

```
double[] a;  
for (int i = 0; i < 9; i++)  
    a[i] = Math.random();
```

Bir dizi oluşturulmadı



Undeclared variable

```
a = new double[10];  
for (int i = 0; i < 10; i++)  
    a[i] = Math.random();
```

Verinin tipi belirtilmedi

S. İki $N \times N$ matrisin çarpımı için kaç çarpma işlemi yapılır?

```
double[][] c = new double[N][N];  
for (int i = 0; i < N; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < N; k++)  
            c[i][j] += a[i][k] * b[k][j];
```

1. N

2. N^2

3. N^3

4. N^4

Diziler: Mini Sınav

S. İki $N \times N$ matrisin çarpımı için kaç çarpma işlemi yapılır?

```
double[][] c = new double[N][N];  
for (int i = 0; i < N; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < N; k++)  
            c[i][j] += a[i][k] * b[k][j];
```

1. N

2. N^2

3. N^3

4. N^4

← İç içe for döngüleri: $N \times N \times N$

Döngüler

Java döngüleri şu bölümlerden oluşur:

- **Sayaç bildirimi**
- **Sonlandırma koşulu**
- **Sayaç**
- **Döngü gövdesi**

Java döngüleri şunlardır:

- while, döngüsü basit bir yineleme sağlar.
- do while, döngü gövdesine en az bir kez girmeyi garanti eder.
- for, **bildirim**, **sonlandırma koşulu** ve **sayaç** bilgisini ; ile ayrılmış vaziyette birlikte sunar.
- forEach, dizi veya koleksiyon boyunca yineleme için uygun bir yol sağlar

```
while (someMethod()) {  
    // iterative logic  
}
```

while veya do / while döngüleri, iterasyonları kontrol etmek için boolean döndüren bir metod çağrısıyla en iyi şekilde kullanılır.

```
int i = 0;  
while (i < 10) {  
    // iterative logic  
    i++;  
}
```

```
int i = 0;  
do {  
    // iterative logic  
    i++;  
} while (i < 10);
```

```
for (int i = 0; i < 10; i++) {  
    // iterative logic  
}
```

```
int i = 0;  
for ( ; i < 10 ; ) {  
    // iterative logic  
    i++;  
}
```

Önerilen bir kullanım değildir.

Döngüleri Kullanarak Dizilerde İşlem Yapmak

Diziler üzerinde döngü ile işlem yapmak:

- **Sonlandırma durumu** için sınır değerini `array.length` kullanarak belirle,
- Bir indis ve bir yineleyici (iterator) kullanarak **dizi değerlerine** eriş,
- Değerleri **otomatik olarak çıkarmak** için `forEach` döngüsü kullan.

```
int[] values = {1,2,3};
StringBuilder txt = new StringBuilder();
for (int i = 0; i < values.length; i++) {
    int value = values[i];
    txt.append(value);
}
for (int value: values) {
    txt.append(value);
}
```

Not: "forEach" gerçek bir operatör veya rezerve bir kelime değil, yalnızca bir indis ve yineleyici mantığını kullanmaya gerek kalmadan dizi veya koleksiyonda bir "for" döngüsünü tanımlayan bir kelimedir.

Karmaşık for Döngüleri

for döngüsünün pozisyonel doğası ile şu şekilde çalışması mümkündür:

- ", " ile ayrılmış çoklu yineleyici tanımlanabilir.
- **Yineleyici** bölümü gerekli **eylemleri** içeriyorsa gövdesiz bir for döngüsü yazabilirsiniz. (Ancak, karışık eylemler ve yineleme mantığı kafa karıştırıcı olabilir ve bu nedenle iyi bir kodlama uygulaması değildir.)

```
int[] values = {1,2,3,4,5,6,7,8,9};  
int sum = 0;  
for (int i = 0 ; i < values.length ; sum += i++) ;  
// sum is 36
```

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};  
for (int i = 0, j = 2 ; !(i == 3 || j == -1) ; i++, j--) {  
    int value = matrix[i][j];  
}
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

İç İçe Döngüler

Döngüler farklı bir döngü içerisinde kullanılabilir:

- Çok boyutlu dizilerdeki işlemler için kullanışlıdır.
- Herhangi bir döngü tipi ile iç içelik kurgulanabilir: while, do-while, for ve for-each

Result:

```
OXX
XXO
O O
OXX
XXO
O O
```

	0	1	2	x
0	O	X	X	
1	X	X	O	
2	O		O	
y				

```
char[][] game = {{'O','X','X'},
                  {'X','X','O'},
                  {'O',' ','O'}};

StringBuilder txt = new StringBuilder();
for (int x = 0; x < game.length; x++) {
    int y = 0;
    while (y < game[x].length) {
        txt.append(game[x][y]);
        y++;
    }
    txt.append('\n');
}
for(char[] row : game) {
    for (char value: row) {
        txt.append(value);
    }
    txt.append('\n');
}
```

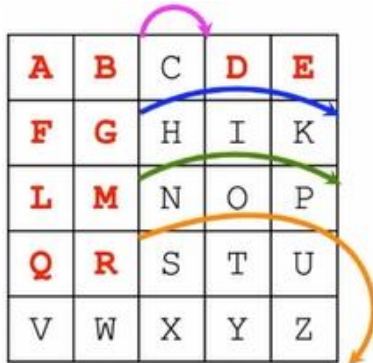
break ve continue

Döngüleri kırma ve döngü çevrimini atlama:

- **continue** operatörü mevcut döngü çevrimini atlar,
- **continue <label>** etiketli döngü çevrimini atlar.
- **break** mevcut döngüyü sonlandırır.
- **break <label>** etiketli döngüyü sonlandırır.

Result:

ABDE
FGLM
QR



A	B	C	D	E
F	G	H	I	K
L	M	N	O	P
Q	R	S	T	U
V	W	X	Y	Z

```
char[][] matrix = {{ 'A', 'B', 'C', 'D', 'E' },  
                   { 'F', 'G', 'H', 'I', 'K' },  
                   { 'L', 'M', 'N', 'O', 'P' },  
                   { 'Q', 'R', 'S', 'T', 'U' },  
                   { 'V', 'W', 'X', 'Y', 'Z' } };  
  
StringBuilder txt = new StringBuilder();  
outerLoopLabel:  
for(char[] row : matrix) {  
    for (char value: row) {  
        if (value == 'C') { continue; }  
        if (value == 'H') { continue outerLoopLabel; }  
        if (value == 'N') { break; }  
        if (value == 'S') { break outerLoopLabel; }  
        txt.append(value);  
    }  
    txt.append('\n');  
}
```

Döngüler: Örnek

S. Aşağıdaki kodda hata var mıdır?

```
public class PQwhile
{
    public static void main(String[] args)
    {
        int n = Integer.parseInt(args[0]);
        int i = 0;
        int v = 1;
        while (i <= n)
            System.out.println(v);
            i = i + 1;
            v = 2 * v;
    }
}
```

Döngüler: Örnek

S. Aşağıdaki kodda hata var mıdır?

```
public class PQwhile
{
    public static void main(String[] args)
    {
        int n = Integer.parseInt(args[0]);
        int i = 0;
        int v = 1;
        while (i <= n)
        {
            System.out.println(v);
            i = i + 1;
            v = 2 * v; }
    }
}
```

Q. Parantezsiz sonuç ne olur?

A. *Sonuz döngüye girer.*

```
% java PQwhile 6
1
1
1
1
1
1
1
1
1
1
1
1
```

A. Evet! Parantez gerekli.

Döngüler: Örnek

```
int sum = 0;
for (int i = 1; i <= N; i++)
    sum += i;
System.out.println(sum);
```

Toplamı hesapla ($1 + 2 + 3 + \dots + N$)

top	i
1	1
3	2
6	3
10	4

$N = 4$ için döngüyü sonlandır

```
long product = 1;
for (int i = 1; i <= N; i++)
    product *= i;
System.out.println(product);
```

$N!$ hesapla $= 1 * 2 * 3 * \dots * N$

carpim	i
1	1
2	2
6	3
24	4

```
for (int k = 0; k <= N; k++)
    System.out.println(k + " " + 2*Math.PI*k/N);
```

Fonksiyon değerlerini hesapla

k	$\frac{2nk}{U}$
0	0
1	1.57079632...
2	3.14159265...
3	4.71238898...
4	6.28318530...

```
int v = 1;
while (v <= N/2)
    v = 2*v;
System.out.println(v);
```

2 nin N . kuvveti

v
2
4
8
16

$N = 4$ için döngüyü sonlandır

Döngüler: Örnek

S. Aşağıdaki programın çıktısı ne olur?

```
public class PQfor
{
    public static void main(String[] args)
    {
        int f = 0, g = 1;
        for (int i = 0; i <= 10; i++)
        {
            System.out.println(f);
            f = f + g;
            g = f - g;
        }
    }
}
```

Döngüler: Örnek

S. Aşağıdaki programın çıktısı ne olur?

```
public class PQfor
{
    public static void main(String[] args)
    {
        int f = 0, g = 1;
        for (int i = 0; i <= 10; i++)
        {
            System.out.println(f);
            f = f + g;
            g = f - g;
        }
    }
}
```

C.

Döngü başlangıcı

i	f	g
0	0	1
1	1	0
2	1	1
3	2	1
4	3	2
5	5	3
6	8	5
7	13	8
8	21	13
9	34	21
10	55	34

↑
Yazdırılan değerler

Döngüler: Örnek

Amaç. Verilen gelire uygun vergi dilimini belirlemek.

gelir	oran
0 – 47,450	22%
47,450 – 114,649	25%
114,650 – 174,699	28%
174,700 – 311,949	33%
311,950 +	35%

```
if (income < 47450) rate = 0.22;  
else  
{
```

```
    if (income < 114650) rate = 0.25;  
    else  
    {
```

```
        if (income < 174700) rate = 0.28;  
        else  
        {
```

```
            if (income < 311950) rate = 0.33;  
            else  
                rate = 0.35;  
        }
```

```
    }
```

```
}
```

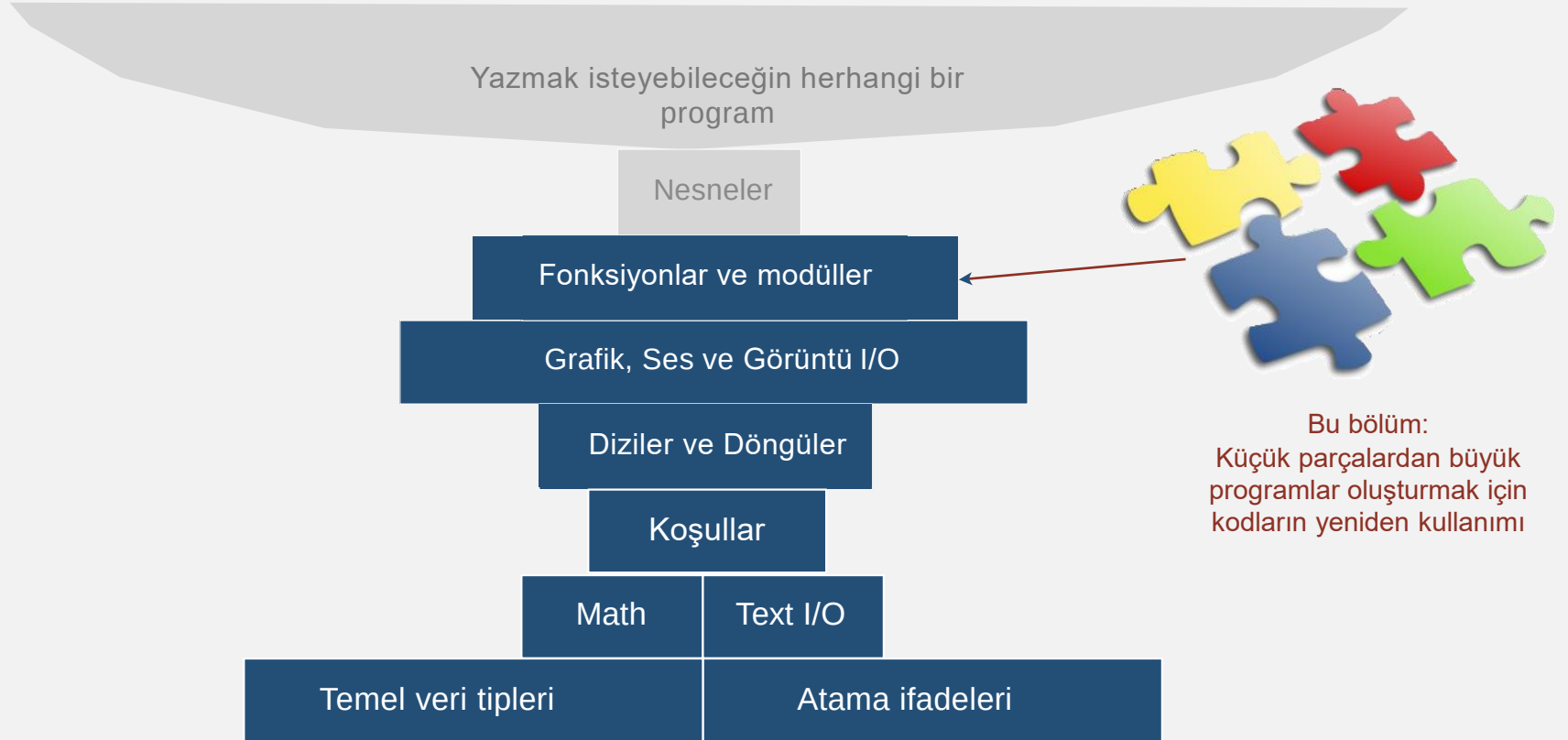
if koşulu
İçindeki if koşulu

if koşulu
İçindeki if koşulu
İçindeki if koşulu

if koşulu
İçindeki if koşulu
İçindeki if koşulu
İçindeki if koşulu

Temel veri tipleri
Koşul ifadeleri
Diziler ve Döngüler
Metotlar

Programlama için temel yapı taşları



Fonksiyonlar, kütüphaneler ve modüller

Modüler programlama

- Programları, birlikte bir işi yapan bağımsız modüller olarak düzenleyin.
- Neden? Daha büyük programlar oluşturmak için kodu paylaşmak ve yeniden kullanmak daha kolaydır.



Tanım. Bir **kütüphane** fonksiyonlar kümesidir.

↑
Bu konunun amacına göre

Tanım. Bir **modül**, bir .java dosyasıdır.

↑
Bu dersin amacına göre

Şimdilik. Kütüphaneler ve modüller *aynı şeydir*: .java dosyaları fonksiyonlar kümesi içerir.

Daha Sonra. Modüller *veri tiplerini* oluşturur.

Fonksiyonlar (static metotlar)

Java *fonksiyon*

- Sıfır veya daha fazla girdi argümanı alır
- Sıfır veya bir çıktı değeri döndürür.

Uygulamalar

- Bilim adamları formülleri hesaplamak için fonksiyonları kullanır.
- Programcılar modüler programları inşa etmek için kullanır.
- Siz fonksiyonları ikisi için de kullanırsınız.

Bildiklerinizden

- Yerleşik fonksiyonlar: `Math.random()`, `Math.abs()`, `Integer.parseInt()`.
- I/O kütüphanemiz: `StdIn.readInt()`, `StdDraw.line()`, `StdAudio.play()`.
- Kullanıcı tanımlı fonksiyonlar: `main()`.



Bir Java static metodun anatomisi

Bir fonksiyonu tanımlamak için (static method)

- Bir *isim* verin
- *Parametre(ler)* için isim ve veri tipi tanımlaması yapın.
- *Return değeri* için tip belirtin.
- Metodun *gövdesini* oluşturun.
- *return* ifadesi ile bitirin.

```
public static double sqrt(double c, double eps)
{
    if (c < 0) return Double.NaN;
    double t = c;
    while (Math.abs(t - c/t) > eps * t)
        t = (c/t + t) / 2.0;
    return t;
}
```

return tipi

metot adı

Parametre tanımlama

Metodun imzası

return ifadesi

sqrt() metodunun gövdesi

Bir Java kütüphanesinin anatomisi

Kütüphane bir fonksiyonlar kümesidir.

Not: Temellerini tanıdık bir işlevle göstermek için burada daha önce bahsedilen `sqrt()` yöntemimizi kullanıyoruz.

`Math.sqrt()` kullanabilirsiniz.

`sqrt()` metodu →

`Newton.java`
İsimli modül →

`main()` metodu →

```
public class Newton ← kütüphane/modül adı
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }

    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
            StdOut.println(sqrt(a[i], 1e-3));
    }
}
```

Kilit Nokta. Fonksiyonlar, yürütme akışını kontrol etmek için *yeni bir yol* sunar.

Scope

Tanım. Bir değişken için **Scope** ona adıyla erişebilen koddur.

```
public class Newton
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }

    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
            StdOut.println(sqrt(a[i], 1e-3));
    }
}
```

scope of c and eps →

scope of t →

scope of a →

a veya i ye erişemez

cannot refer to c, eps, or t in this code

Bir Java kütüphanesinde, bir değişkenin scope u, aynı blokta tanımlanmasını izleyen koddur.

her birinin kapsamı tek bir for döngüsü ile sınırlı olan i adında iki farklı değişken

Kontrol akışı

```
public class Newton
{
    public static double sqrt(double c, double eps)
    {
        if (c < 0) return Double.NaN;
        double t = c;
        while (Math.abs(t - c/t) > eps * t)
            t = (c/t + t) / 2.0;
        return t;
    }
    public static void main(String[] args)
    {
        double[] a = new double[args.length];
        for (int i = 0; i < args.length; i++)
            a[i] = Double.parseDouble(args[i]);
        for (int i = 0; i < a.length; i++)
        {
            double x = sqrt(a[i], 1e-3);
            StdOut.println(x);
        }
    }
}
```

Bir fonksiyon çağrısı için özet kontrol akışı

- Kontrol fonksiyon koduna aktarılır.
- Argüman değişkenleri tanımlanır ve başlangıç değerleri atanır.
- Fonksiyon kodu çalıştırılır.
- Kontrol çağırان koda geri döner.
(çağırان koddaki fonksiyon adına geri dönüş değerinin atamasıyla)

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1a
{
    public static int cube(int i)
    {
        int j = i * i * i;
        return j;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1a
{
    public static int cube(int i)
    {
        int j = i * i * i;
        return j;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

C. Komut satırından N değerini alır, sonra 1 den N e kadar sayıların küplerini ekrana yazdırır.

```
% javac PQfunctions1a.java
% java PQfunctions1a 6
1 1
2 8
3 27
4 64
5 125
6 216
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1b
{
    public static int cube(int i)
    {
        int i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1b
{
    public static int cube(int i)
    {
        int i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Derlenmeyecektir. *i* değişkeni fonksiyon parametresi olarak tanımlandığı için kod bloğunda tekrar kullanılamaz.

```
% javac PQfunctions1b.java
PQfunctions1b.java:5: i is already defined in cube(int)
        int i = i * i * i;
            ^
1 error
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1c
{
    public static int cube(int i)
    {
        i = i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQ6_1c
{
    public static int cube(int i)
    {
        i = i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Çalışmayacaktır. Fonksiyon sonucu döndürülmelidir.

```
% javac PQfunctions1c.java
PQfunctions1c.java:6: missing return statement
    }
    ^
1 error
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1d
{
    public static int cube(int i)
    {
        i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1d
{
    public static int cube(int i)
    {
        i = i * i * i;
        return i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

A. Çalışır. cube()içindeki i:

- Tanımlanır ve bir fonksiyon parametresi olarak başlatılır.
- main() içinde kullanılan i den farklıdır.

Fonksiyon parametresi olarak tanımlanan değişken değerini değiştirmek kötü bir kullanımdır.

```
% javac PQfunctions1d.java
% java PQfunctions1d 6
1 1
2 8
3 27
4 64
5 125
6 216
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1e
{
    public static int cube(int i)
    {
        return i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

S. Aşağıdaki kodu derleyip çalıştırdığınızda ne olur?

```
public class PQfunctions1e
{
    public static int cube(int i)
    {
        return i * i * i;
    }
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        for (int i = 1; i <= N; i++)
            StdOut.println(i + " " + cube(i));
    }
}
```

C. Doğru çalışır. Tercih edilen kod kullanımıdır.

```
% javac PQfunctions1e.java
% java PQfunctions1e 6
1 1
2 8
3 27
4 64
5 125
6 216
```

SON